INT4211 – LẬP TRÌNH TRÊN CÁC THIẾT BỊ DI ĐỘNG

Đồ án cuối kỳ · HK261

CHỐT PHÂN TÍCH – MỐC TUẦN 9

Sơ đồ use case · Sơ đồ cơ sở dữ liệu · Bảng phân công (bản nháp)

. Hệ thống quản lý chuỗi quán cà phê – trà sữa

Nhóm	**Nhóm 36**
Repository GitHub	
Thành viên 1 (P1)	
Thành viên 2 (P2)	
Thành viên 3 (P3)	
Thành viên 4 (P4)	

1. Giới thiệu

1.1. Bối cảnh và vấn đề

TP.HCM có hàng chục nghìn điểm bán cà phê, trà sữa. Phần lớn chuỗi nhỏ 3–10 chi nhánh vẫn ghi đơn bằng giấy và chốt doanh thu bằng Excel. Chủ chuỗi cần xem được doanh thu từng chi nhánh theo ca ngay trên điện thoại; khách muốn tự gọi món bằng QR tại bàn để khỏi xếp hàng.

Nhóm xây dựng **một hệ thống** gồm hai ứng dụng dùng chung một máy chủ và một cơ sở dữ liệu: app quản trị cho chủ chuỗi / quản lý ca và app gọi món cho khách.

1.2. Kiến trúc và công nghệ

Thành phần	Công nghệ	Vai trò
App Admin	Android (Java + XML), Retrofit 2	Chủ chuỗi / quản lý ca: thực đơn, nhận đơn, bàn & QR, mở–chốt ca, báo cáo, khuyến mãi, nhân viên
App Client	Flutter 3.35+ (Dart 3)	Khách: quét QR, xem thực đơn, tuỳ chọn món, giỏ hàng, theo dõi đơn, tích điểm, đánh giá
Backend	Node.js 20 + Express 4, JWT + bcrypt, thư viện pg (SQL thuần)	REST API, phân quyền theo vai trò, tính tiền phía máy chủ; deploy trên Render
Cơ sở dữ liệu	PostgreSQL 15 trên Supabase	19 bảng, 30 khoá ngoại; kết nối qua Connection pooler (Session mode, cổng 5432)

Nguyên tắc xuyên suốt: mọi quyết định nghiệp vụ nằm ở backend. App chỉ gửi *lựa chọn* (mã món, tuỳ chọn, topping, số lượng, mã giảm giá) và hiển thị kết quả; **không bao giờ gửi giá**. Sửa giá trên điện thoại không làm thay đổi số tiền thật của đơn — đây là điểm nhấn kỹ thuật của đề.

1.3. Phạm vi KHÔNG làm

Hệ thống tập trung vào nghiệp vụ bán hàng tại quầy của một chuỗi 3–10 chi nhánh và **không** bao gồm: website quản trị (toàn bộ thao tác quản lý chỉ có trên app Android); tích hợp cổng thanh toán hoặc API ngân hàng (chuyển khoản chỉ được **ghi nhận thủ công** và đối chiếu khi chốt ca, hệ thống không xác nhận được tiền đã vào tài khoản hay chưa); giao hàng tận nơi, đặt mang đi và đặt trước; quản lý kho nguyên liệu, định lượng công thức và nhà cung cấp; in hoá đơn bằng máy in nhiệt; thông báo đẩy (push notification) — đơn mới chỉ báo bằng âm thanh và badge **trong app** đang mở; chế độ offline (app yêu cầu có mạng); upload ảnh món từ app (ảnh dùng URL có sẵn); đa ngôn ngữ, đa tiền tệ; phân tích dự báo bằng AI; nhật ký thao tác chi tiết (audit log), sửa/xoá đánh giá, hoàn tiền trực tuyến và chương trình khách hàng thân thiết nhiều hạng. Ứng dụng Flutter về lý thuyết build được cho iOS nhưng đồ án **chỉ kiểm thử trên Android**; hệ thống cũng chỉ phục vụ **một chuỗi** duy nhất, không phải nền tảng SaaS cho nhiều chuỗi thuê chung.

2. Phân tích yêu cầu

2.1. Tác nhân và quan hệ

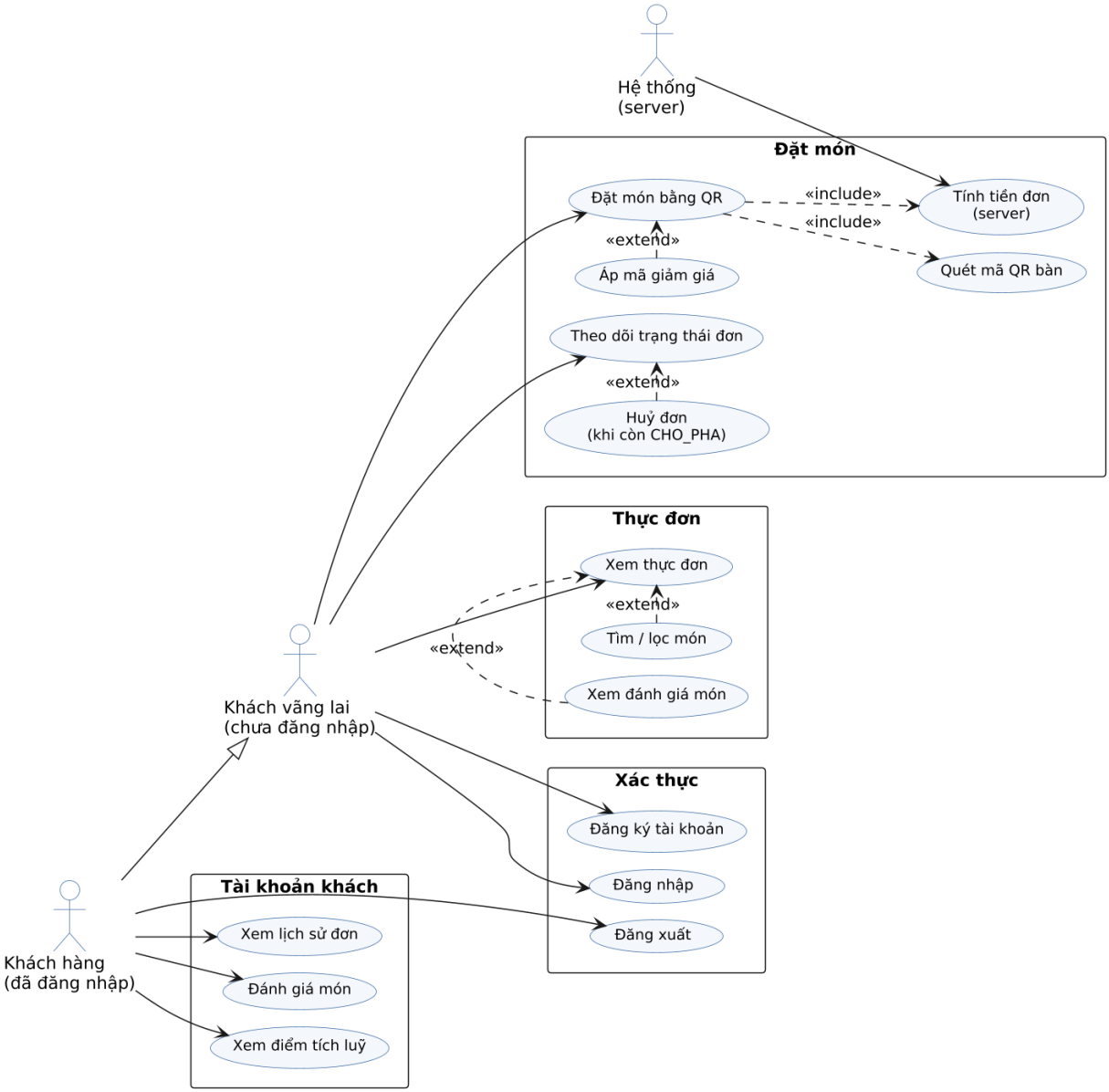
Tác nhân	Mô tả	Quyền hạn
Khách vãng lai	Người quét QR tại bàn, chưa có tài khoản	Xem thực đơn, chọn món, đặt đơn, theo dõi đơn, áp mã giảm giá
Khách hàng	Khách đã đăng ký/đăng nhập	*Kế thừa toàn bộ* Khách vãng lai + tích điểm, lịch sử đơn, đánh giá
Quản lý ca	Nhân viên quản lý một chi nhánh	Chỉ thao tác trên dữ liệu **chi nhánh mình**: thực đơn, đơn hàng, bàn/QR, ca làm, báo cáo chi nhánh
Chủ chuỗi	Chủ hệ thống 3–10 chi nhánh	*Kế thừa toàn bộ* Quản lý ca + thao tác trên **mọi chi nhánh**: khuyến mãi, nhân viên, so sánh báo cáo
Hệ thống (server)	Tác nhân phụ, không phải người	Tính tiền, cộng điểm — người dùng không gọi trực tiếp, chỉ gián tiếp qua use case khác

Quan hệ kế thừa: Khách hàng ──▷ Khách vãng lai, Chủ chuỗi ──▷ Quản lý ca.

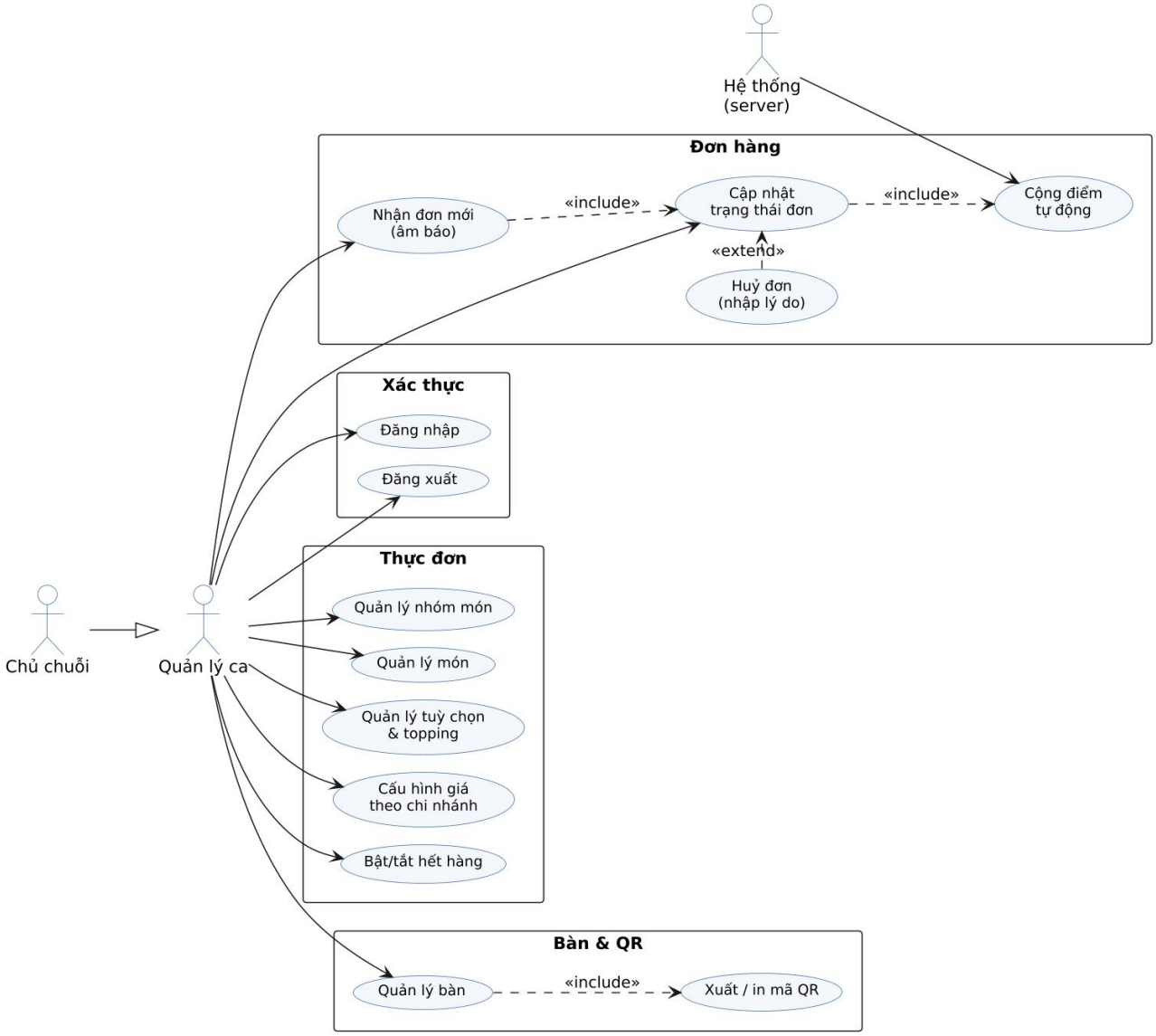
Nhờ kế thừa, Chủ chuỗi **không cần** vẽ lại các đường nối tới use case của Quản lý ca.

2.2. Sơ đồ use case

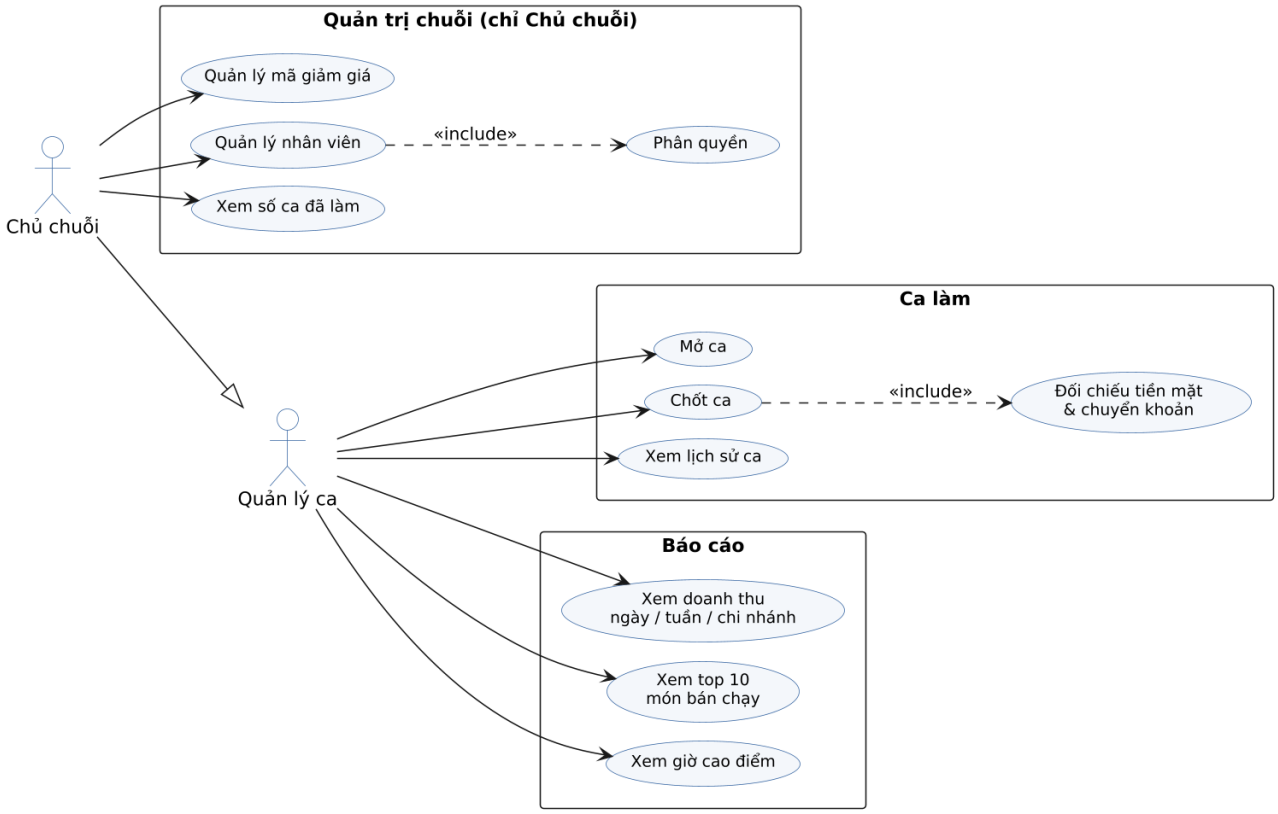
Để sơ đồ đủ rõ trên khổ giấy, hệ thống được vẽ thành ba hình theo nhóm tác nhân. Quan hệ kế thừa: Khách hàng kế thừa Khách vãng lai; Chủ chuỗi kế thừa Quản lý ca (nên không vẽ lại các đường nối của Quản lý ca cho Chủ chuỗi).



Hình 1. Use case phía khách (Khách vãng lai, Khách hàng, Hệ thống)



Hình 2. Use case vận hành tại quầy: thực đơn, đơn hàng, bàn & QR (Quản lý ca, Chủ chuỗi)



Hình 3. Use case ca làm, báo cáo và quản trị chuỗi (Quản lý ca, Chủ chuỗi)

2.3. Danh sách use case theo module

Module	Use case	Tác nhân	Quan hệ
Xác thực	Đăng nhập	Khách vãng lai, Quản lý ca*	—
	Đăng ký tài khoản	Khách vãng lai	—
	Đăng xuất	Khách hàng, Quản lý ca*	—
Thực đơn	Xem thực đơn	Khách vãng lai*	bị extend bởi Tìm món, Xem đánh giá
	Tìm / lọc món	Khách vãng lai*	extend Xem thực đơn
	Quản lý nhóm món	Quản lý ca*	—
	Quản lý món	Quản lý ca*	—
	Quản lý tuỳ chọn & topping	Quản lý ca*	—
	Cấu hình giá theo chi nhánh	Quản lý ca*	—
	Bật/tắt hết hàng	Quản lý ca*	—
Đơn hàng	Đặt món bằng QR	Khách vãng lai*	include Quét QR, Tính tiền; bị extend bởi Áp mã
	Tính tiền đơn	Hệ thống	được include
	Theo dõi trạng thái đơn	Khách vãng lai*	bị extend bởi Huỷ đơn
	Nhận đơn mới (âm báo)	Quản lý ca*	include Cập nhật trạng thái
	Cập nhật trạng thái đơn	Quản lý ca*	include Cộng điểm
	Huỷ đơn	Khách vãng lai*, Quản lý ca*	extend Theo dõi / Cập nhật trạng thái
	Xem lịch sử đơn	Khách hàng	—
Bàn & QR	Quét mã QR bàn	Khách vãng lai*	được include bởi Đặt món

Module	Use case	Tác nhân	Quan hệ
	Quản lý bàn	Quản lý ca*	include Xuất/in QR
	Xuất / in mã QR	Quản lý ca*	được include
Ca làm	Mở ca	Quản lý ca*	—
	Chốt ca	Quản lý ca*	include Đối chiếu tiền
	Đối chiếu tiền mặt & chuyển khoản	Quản lý ca*	được include
	Xem lịch sử ca	Quản lý ca*	—
Báo cáo	Xem doanh thu ngày/tuần/chi nhánh	Quản lý ca*	—
	Xem top 10 món bán chạy	Quản lý ca*	—
	Xem giờ cao điểm	Quản lý ca*	—
Khuyến mãi	Quản lý mã giảm giá	Chủ chuỗi	—
	Áp mã giảm giá	Khách vãng lai*	extend Đặt món
Nhân viên	Quản lý nhân viên	Chủ chuỗi	include Phân quyền
	Phân quyền	Chủ chuỗi	được include
	Xem số ca đã làm	Chủ chuỗi	—
Đánh giá	Đánh giá món	Khách hàng	—
	Xem đánh giá món	Khách vãng lai*	extend Xem thực đơn
Tích điểm	Xem điểm tích luỹ	Khách hàng	—
	Cộng điểm tự động	Hệ thống	được include bởi Cập nhật trạng thái

* = tác nhân kế thừa cũng thực hiện được (Khách hàng thay Khách vãng lai, Chủ chuỗi thay Quản lý ca).

2.4. Đặc tả 7 use case quan trọng nhất

UC-01. Đặt món bằng QR

Tác nhân	Khách vãng lai (chính), Khách hàng
Điều kiện trước	Bàn đang ở trạng thái sử dụng; chi nhánh đang hoạt động và **đang có ca mở**
Kết quả	Đơn hàng CHO_PHA được tạo trong DB, tổng tiền do **server** tính, khách nhận mã đơn để theo dõi

Luồng chính

1. Khách quét mã QR tại bàn. App gửi nội dung QR (UUID bàn) lên server.
2. Server tra bàn → suy ra chi nhánh, kiểm tra bàn đang dùng và chi nhánh có ca mở.
3. Server trả thực đơn của đúng chi nhánh: nhóm món, món, giá riêng của chi nhánh, cờ hết hàng, tuỳ chọn và topping áp dụng cho từng món.
4. Khách chọn món → chọn size / mức đường / mức đá / topping, số lượng. App gửi lên **lựa chọn**, tuyệt đối không gửi giá: {ban_id, mon:[{mon_id, tuy_chon_id[], topping_id[], so_luong}], phuong_thuc_tt, ma_client}.
5. Server kiểm tra từng dòng: món có bán tại chi nhánh, không hết hàng, tuỳ chọn thuộc đúng món, số lượng 1–99.
6. Server tính tiền (UC-02 nếu có mã giảm giá), trả về tổng để app hiển thị.
7. Khách bấm xác nhận.

8. Server tính lại tiền từ đầu, tạo don_hang + chi_tiet_don (lưu **snapshot** tên món, đơn giá, tuỳ chọn), gắn ca_lam_id của ca đang mở, trả ma_don + thanh_tien.
9. App chuyển sang màn theo dõi trạng thái, tự cập nhật mỗi 5 giây.

Luồng thay thế / ngoại lệ

- 2a. QR không hợp lệ hoặc bàn bị khoá → 400 BAN_KHONG_HOP_LE, app yêu cầu quét lại.
- 2b. Chi nhánh chưa mở ca → 409 CHUA_MO_CA, app báo "quán chưa mở ca, vui lòng gọi nhân viên". Đơn không được tạo để không làm lệch số liệu chốt ca.
- 5a. Món hết hàng / ngừng bán tại chi nhánh → 400, app đánh dấu đỏ đúng dòng món, khách bỏ hoặc thay món.
- 5b. tuy_chon_id không thuộc món (client bị sửa) → 400 TUY_CHON_KHONG_HOP_LE.
- 7a. Mất mạng khi xác nhận → app gửi lại với **cùng ma_client**; server có UNIQUE trên ma_client nên trả lại đơn cũ thay vì tạo đơn trùng.
- 8a. Request có kèm field gia / thanh_tien → server **bỏ qua hoàn toàn**, dùng giá trong DB. Đây là yêu cầu bắt buộc của đề.

UC-02. Áp mã giảm giá (extend UC-01)

Tác nhân	Khách vãng lai, Khách hàng
Điều kiện trước	Giỏ có ≥ 1 món; mã tồn tại và đang bật
Kết quả	tien_giam và thanh_tien được server tính và lưu; lượt dùng mã tăng 1

Luồng chính

1. Khách nhập mã ở màn giỏ hàng; app gửi {ma, ban_id}.
2. Server kiểm tra: mã tồn tại, đang bật, trong hạn, đúng chi nhánh (nếu mã giới hạn chi nhánh), còn lượt, tạm tính ≥ đơn tối thiểu.
3. Server tính giảm:
   - PERCENT: min(tạm_tính × giá_trị / 100, giảm_tối_đa) (nếu có giảm tối đa)
   - AMOUNT: min(giá_trị, tạm_tính)
   - Làm tròn xuống bội số 1.000đ.
4. Server trả {hop_le, so_tien_giam, thanh_tien}. App chỉ hiển thị, không tự tính.
5. Khi khách bấm đặt đơn, server tính lại lần nữa và UPDATE ma_giam_gia SET da_dung = da_dung + 1 WHERE id = ? AND da_dung < so_luot trong cùng transaction với việc tạo đơn.

Luồng thay thế / ngoại lệ

- 2a. Mã hết hạn → 400 MA_HET_HAN; hết lượt → 400 MA_HET_LUOT; dưới mức tối thiểu → 400 CHUA_DU_DIEU_KIEN; mã của chi nhánh khác → 400 MA_KHONG_AP_DUNG.
- 3a. Số tiền giảm lớn hơn tạm tính → kẹp bằng tạm tính, thanh_tien = 0, **không bao giờ âm**.
- 5a. Hai khách cùng dùng lượt cuối: câu UPDATE có điều kiện trả 0 dòng → rollback, trả 400 MA_HET_LUOT cho người đến sau.

UC-03. Cập nhật trạng thái đơn (kèm âm báo đơn mới)

Tác nhân	Quản lý ca (chỉ chi nhánh mình)
Điều kiện trước	Đơn tồn tại và thuộc chi nhánh của người dùng
Kết quả	Trạng thái đơn và thời điểm hoàn tất được ghi; khách thấy cập nhật trong ≤ 5 giây

Luồng chính

1. App admin gọi GET /api/don-hang?chi_nhanh_id=&trang_thai=CHO_PHA mỗi 3 giây.
2. App so sánh danh sách id với lần trước; có id mới → phát âm báo + tăng badge.
3. Quản lý bấm "Bắt đầu pha" → PATCH /api/don-hang/:id/trang-thai {trang_thai: DANG_PHA}.
4. Server kiểm tra transition hợp lệ (CHO_PHA → DANG_PHA) và quyền chi nhánh, ghi trạng thái.
5. Pha xong → HOAN_TAT: server ghi hoan_tat_luc, cộng điểm cho khách nếu đơn có khach_id (1 điểm / 10.000đ).
6. App khách đang theo dõi poll thấy trạng thái mới và hiển thị.

Luồng thay thế / ngoại lệ

- 4a. Nhảy cóc CHO_PHA → HOAN_TAT → 400 TRANSITION_KHONG_HOP_LE.
- 4b. Đơn đã HOAN_TAT hoặc HUY → 409 DON_DA_KET_THUC.
- 4c. Quản lý chi nhánh A thao tác đơn chi nhánh B → 403.
- 3a. Huỷ đơn (extend): chỉ từ CHO_PHA hoặc DANG_PHA, bắt buộc nhập lý do; khách chỉ huỷ được khi đơn còn CHO_PHA.
- Âm báo: nếu app bị đóng/ngủ, đơn vẫn còn trong DB — mở lại app sẽ thấy ngay, không mất đơn.

UC-04. Chốt ca

Tác nhân	Quản lý ca
Điều kiện trước	Chi nhánh đang có đúng **một** ca mở của người dùng này; không còn đơn CHO_PHA/DANG_PHA trong ca
Kết quả	ca_lam chuyển DA_DONG, lưu tiền thực tế và chênh lệch, **không sửa được nữa**

Luồng chính

1. Quản lý mở màn chốt ca; app gọi GET /api/ca-lam/hien-tai để lấy số liệu tạm tính.
2. Server tính từ DB (không tin số app gửi):
   - doanh_thu_ca = SUM thanh_tien của đơn HOAN_TAT có ca_lam_id = ca này
   - tien_mat_ky_vong = tien_dau_ca + SUM đơn TIEN_MAT
   - chuyen_khoan = SUM đơn CHUYEN_KHOAN
   - số đơn hoàn tất, số đơn huỷ, tổng tiền giảm giá đã áp
3. Quản lý nhập **tiền mặt thực tế trong két** và **tiền chuyển khoản thực tế**.
4. Server tính chenh_lech = tien_mat_thuc_te − tien_mat_ky_vong, lưu vào ca_lam, đặt trang_thai = DA_DONG, dong_luc = now().
5. App hiển thị biên bản chốt ca (doanh thu, tiền mặt, chuyển khoản, chênh lệch) để đối chiếu.

Luồng thay thế / ngoại lệ

- 1a. Còn đơn chưa kết thúc → 409 CON_DON_CHUA_KET_THUC kèm danh sách mã đơn; bắt buộc xử lý (hoàn tất/huỷ) trước khi chốt, để tiền mặt kỳ vọng không bị thiếu.
- 2a. Ca đã đóng → 409 CA_DA_DONG; mọi request sửa ca đã đóng đều bị từ chối.
- 3a. Nhập số âm / không phải số nguyên → 400.
- 4a. Chênh lệch khác 0 vẫn cho chốt nhưng app hiển thị cảnh báo đỏ và ghi vào biên bản để chủ chuỗi đối chiếu.
- Đơn huỷ **không** được cộng vào tiền mặt kỳ vọng (điều kiện trang_thai = 'HOAN_TAT').

UC-05. Cấu hình giá món theo chi nhánh

Tác nhân	Quản lý ca (chi nhánh mình), Chủ chuỗi (mọi chi nhánh)
Điều kiện trước	Đã đăng nhập; món tồn tại
Kết quả	Dòng mon_chi_nhanh được tạo/cập nhật; đơn tạo sau dùng giá mới, đơn cũ giữ nguyên

Luồng chính

1. Quản lý chọn chi nhánh → chọn nhóm món → chọn món.
2. Nhập giá bán tại chi nhánh và bật/tắt "còn hàng".
3. App gọi PUT /api/mon/:id/chi-nhanh/:cnid {gia_co_ban, het_hang}.
4. Server kiểm tra quyền: QUAN_LY chỉ được với chi_nhanh_id của chính mình; CHU_CHUOI được mọi chi nhánh.
5. Server kiểm tra giá là số nguyên, $0 \le$ giá $\le 10.000.000$.
6. Server UPSERT vào mon_chi_nhanh, trả dữ liệu mới.
7. App hiển thị lại danh sách; app khách lấy giá mới ở lần gọi thực đơn kế tiếp.

Luồng thay thế / ngoại lệ

- 4a. Quản lý sửa chi nhánh khác → 403.
- 5a. Giá âm hoặc có phần lẻ → 400 GIA_KHONG_HOP_LE.
- 6a. Món chưa có dòng ở chi nhánh này → tạo mới; món **không có** dòng mon_chi_nhanh sẽ **không hiện** trong thực đơn của chi nhánh đó (đây là cách "không bán món này ở chi nhánh này").
- Bật "hết hàng" chỉ chặn đặt mới, không xoá món khỏi thực đơn (vẫn hiện mờ).

UC-06. Đánh giá món

Tác nhân	Khách hàng
Điều kiện trước	Đã đăng nhập; có đơn chứa món đó ở trạng thái HOAN_TAT
Kết quả	Đánh giá được lưu; điểm trung bình của món thay đổi

Luồng chính

1. Khách mở lịch sử đơn → chọn đơn đã hoàn tất.
2. Chọn món cần đánh giá, chọn 1–5 sao, nhập nhận xét (≤ 500 ký tự).
3. App gọi POST /api/danh-gia {don_hang_id, mon_id, so_sao, nhan_xet}.
4. Server kiểm tra: đơn thuộc đúng khách, trạng thái HOAN_TAT, món có trong đơn, chưa đánh giá món này trong đơn này.
5. Server lưu danh_gia, trả điểm trung bình và số lượt mới của món.
6. Thực đơn của chi nhánh hiển thị sao trung bình cạnh tên món.

Luồng thay thế / ngoại lệ

- 4a. Đơn chưa hoàn tất → 400 DON_CHUA_HOAN_TAT.
- 4b. Đánh giá trùng (UNIQUE don_hang_id + mon_id) → 409 DA_DANH_GIA.
- 4c. so_sao ngoài 1–5 → 400; nhận xét quá dài → 400.
- 4d. Đơn của người khác → 403.
- Không cho sửa/xoá đánh giá trong phạm vi đồ án (ghi rõ ở mục phạm vi).

UC-07. Xem báo cáo doanh thu

Tác nhân	Quản lý ca (chi nhánh mình), Chủ chuỗi (mọi chi nhánh)
Điều kiện trước	Đã đăng nhập
Kết quả	Bảng + biểu đồ doanh thu read-only, tính trực tiếp từ DB

Luồng chính

1. Người dùng chọn khoảng thời gian, chi nhánh (Chủ chuỗi) và cách nhóm: ngày / tuần.
2. App gọi GET /api/bao-cao/doanh-thu?tu=&den=&chi_nhanh_id=&theo=ngay.
3. Server kiểm tra quyền theo chi nhánh, kiểm tra tu ≤ den và khoảng ≤ 366 ngày.
4. Server query đơn HOAN_TAT, nhóm theo date_trunc('day', hoan_tat_luc AT TIME ZONE 'Asia/Ho_Chi_Minh'), trả doanh thu, số đơn, giá trị đơn trung bình.
5. App vẽ biểu đồ cột + bảng tổng.
6. Cùng bộ lọc, người dùng chuyển sang tab "Top 10 món" / "Giờ cao điểm" (use case riêng, cùng tham số).

Luồng thay thế / ngoại lệ

- 3a. tu > den hoặc khoảng quá 366 ngày → 400.
- 3b. Quản lý ca truyền chi_nhanh_id của chi nhánh khác → 403 (server tự gán chi nhánh của họ).
- 4a. Không có đơn nào → trả mảng rỗng; app hiện "Chưa có dữ liệu trong khoảng này", không báo lỗi.
- 4b. Đơn huỷ và đơn chưa hoàn tất **không** được tính vào doanh thu.
- Báo cáo không lưu cache/bảng tổng hợp — dữ liệu đồ án nhỏ, query trực tiếp luôn đúng.

3. Thiết kế cơ sở dữ liệu

3.1. Nguyên tắc thiết kế

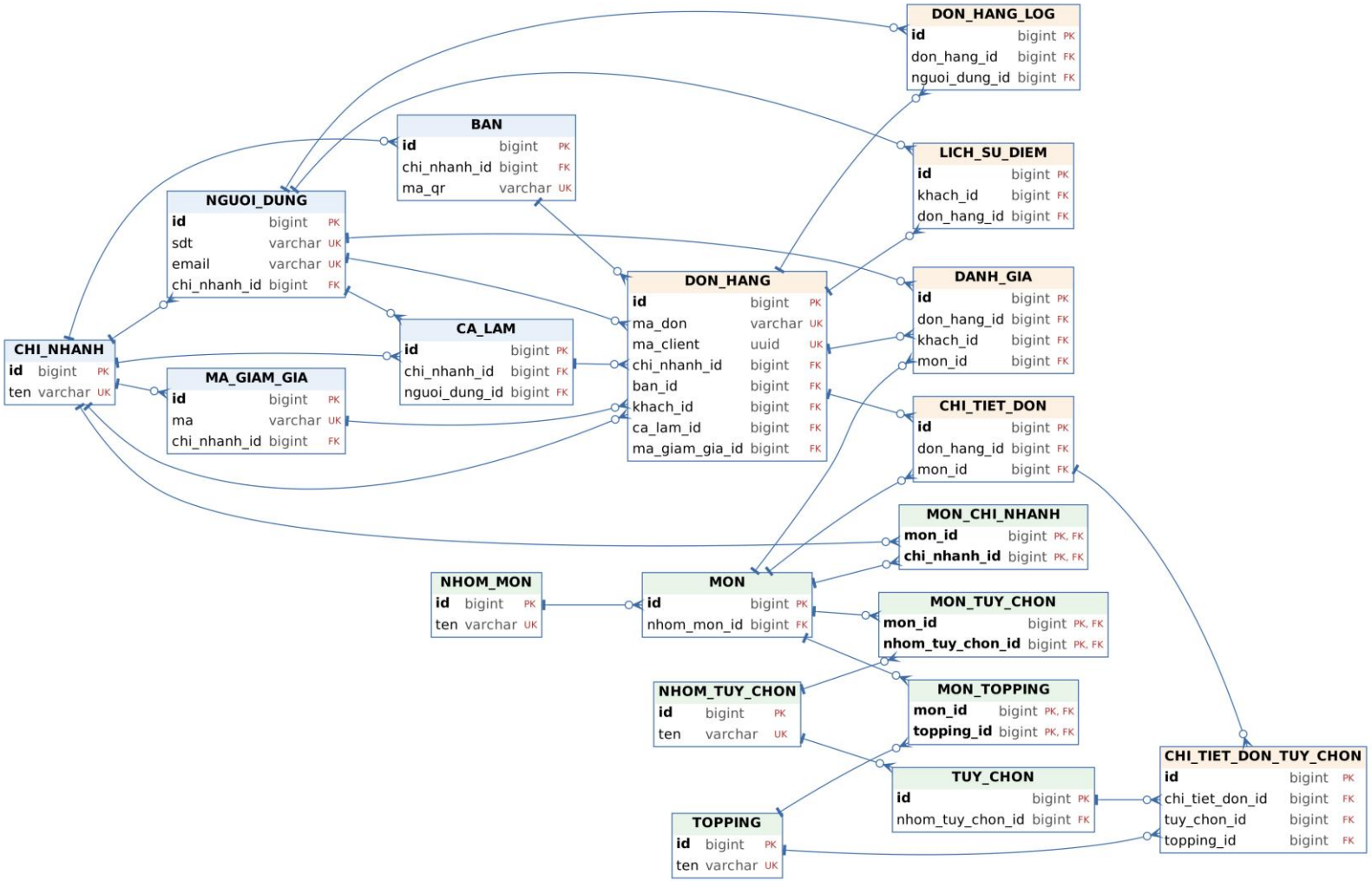
- **19 bảng, 30 khoá ngoại** trên PostgreSQL 15 (Supabase); tệp db/schema.sql chạy lại được từ đầu.
- Mọi cột tiền là BIGINT, đơn vị **đồng** — không dùng float để không lệch 1đ khi chốt ca.
- Mọi mốc thời gian là TIMESTAMPTZ (UTC); báo cáo theo ngày chuyển sang múi giờ Asia/Ho_Chi_Minh.
- Trạng thái dùng CHECK thay vì ENUM để thêm giá trị không phải ALTER TYPE.
- Bất biến quan trọng nằm ở tầng CSDL (CHECK, UNIQUE, partial unique index), không chỉ ở code: mỗi chi nhánh chỉ có một ca mở; quản lý ca bắt buộc thuộc một chi nhánh; không vượt lượt mã giảm giá; thanh_tien = tong_tien_hang − tien_giam ≥ 0.
- Đơn hàng là chứng từ bất biến: chi_tiet_don và chi_tiet_don_tuy_chon lưu **snapshot** tên và giá tại thời điểm mua, nên đổi giá thực đơn không làm sai hoá đơn cũ.

Các bảng được chia làm ba nhóm (xem Hình 4–7):

Nhóm	Bảng	SL
Vận hành	chi_nhanh, nguoi_dung, ban, ca_lam, ma_giam_gia	5
Thực đơn	nhom_mon, mon, mon_chi_nhanh, nhom_tuy_chon, tuy_chon, mon_tuy_chon, topping, mon_topping	8
Giao dịch	don_hang, chi_tiet_don, chi_tiet_don_tuy_chon, danh_gia, lich_su_diem, don_hang_log	6

3.2. Sơ đồ quan hệ thực thể (ERD) tổng quan

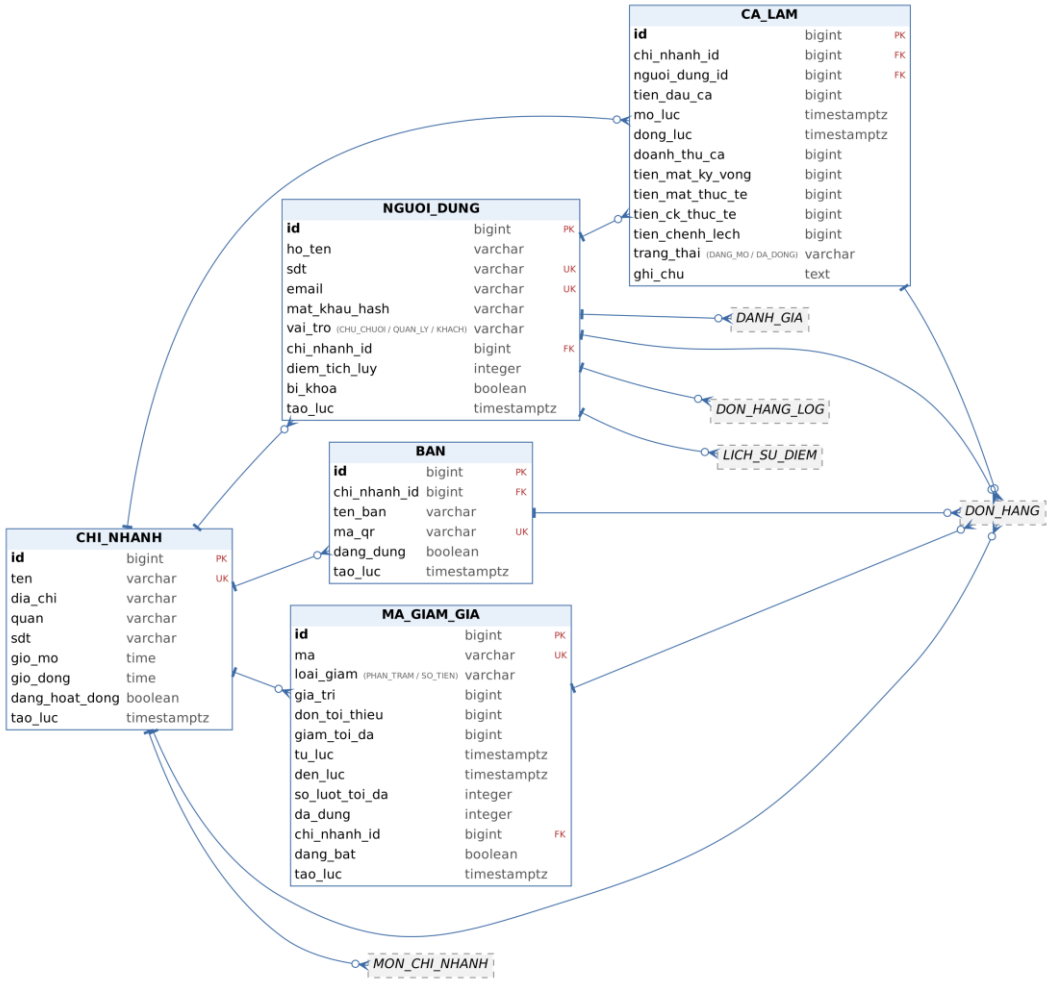
Hình dưới chỉ hiện khoá chính (PK), khoá ngoại (FK) và cột duy nhất (UK) của cả 19 bảng; ký hiệu chân quạ có vòng tròn là “không hoặc nhiều”, vạch ngang là “đúng một”. Màu nền theo nhóm bảng.



Hình 4. ERD tổng quan — 19 bảng, 30 khoá ngoại (chỉ hiện cột khoá)

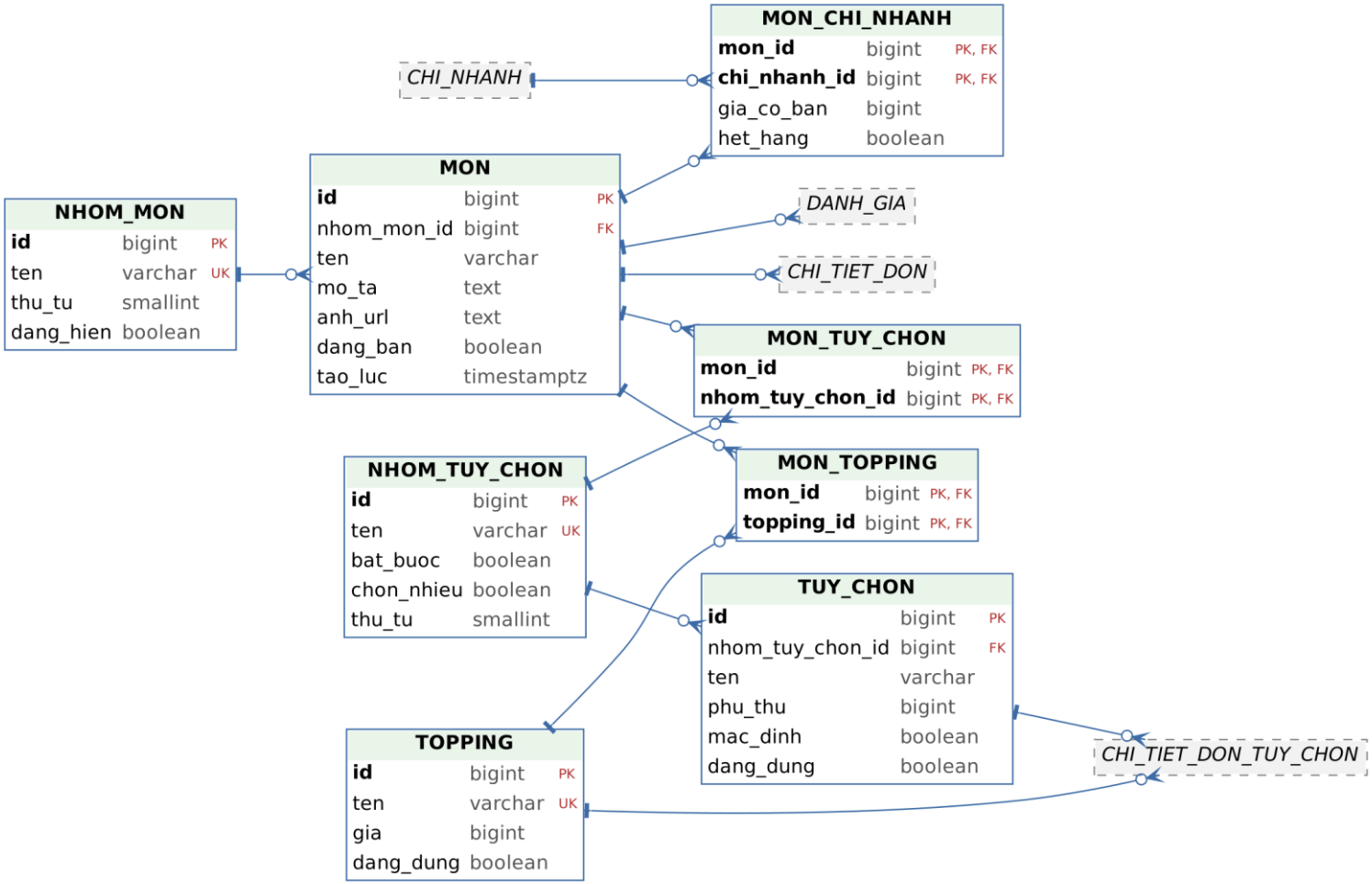
3.3. ERD chi tiết theo nhóm

Nhóm 1 – Vận hành: chi nhánh, nhân sự, bàn, ca làm, khuyến mãi



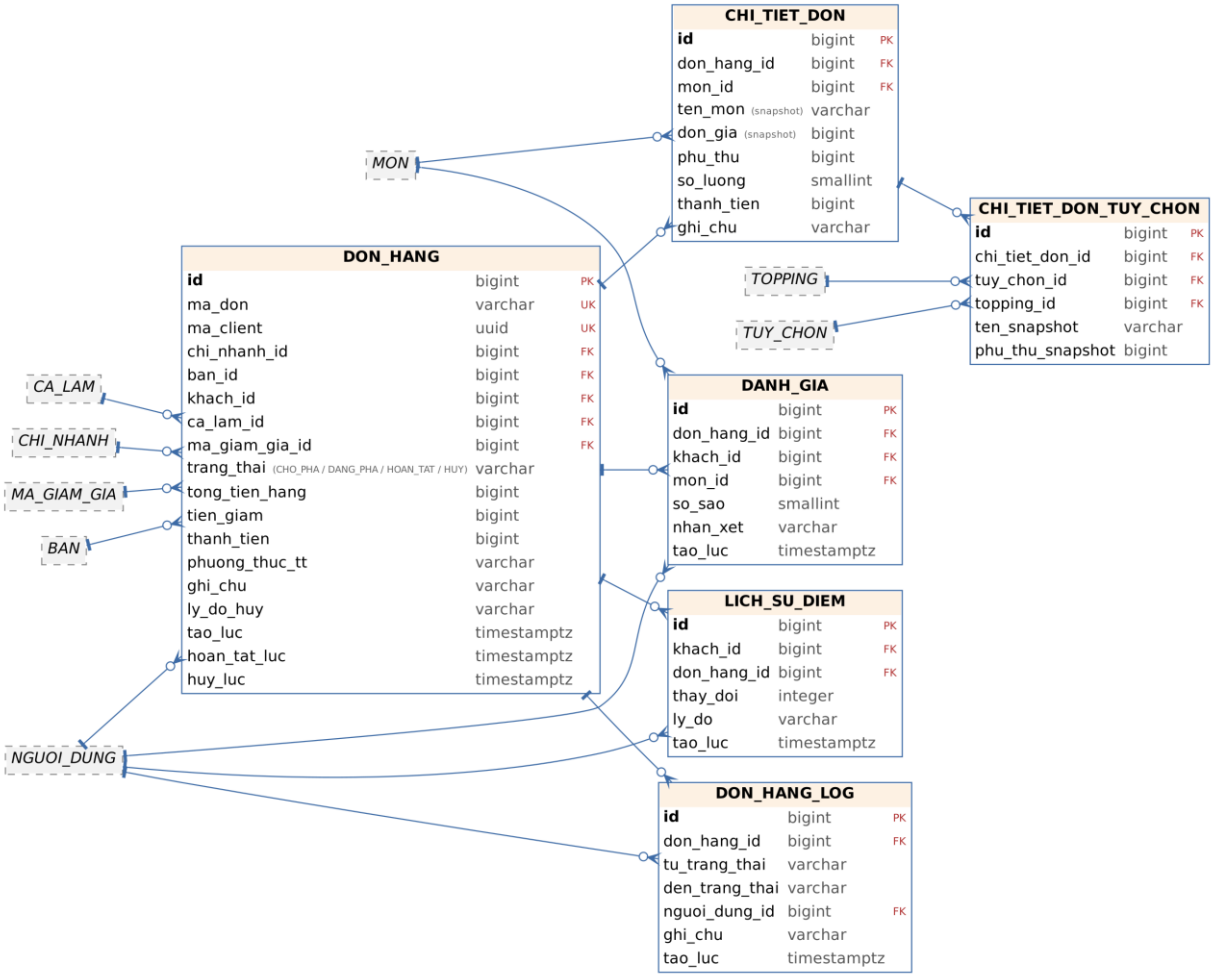
Hình 5. ERD chi tiết nhóm Vận hành (hộp nét đứt là bảng thuộc nhóm khác)

Nhóm 2 – Thực đơn: món, giá theo chi nhánh, tuỳ chọn, topping



Hình 6. ERD chi tiết nhóm Thực đơn

Nhóm 3 – Giao dịch: đơn hàng, chi tiết, đánh giá, điểm, nhật ký



Hình 7. ERD chi tiết nhóm Giao dịch

3.4. Mô tả từng bảng và ràng buộc

3.4.1 chi_nhanh — 1 dòng / cửa hàng

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
ten	VARCHAR(100)	NOT NULL, UNIQUE
dia_chi	VARCHAR(255)	NOT NULL
quan	VARCHAR(50)	NOT NULL (Quận 1, Thủ Đức…)
sdt	VARCHAR(15)	CHECK ^0[0-9]{9,10}$
gio_mo / gio_dong	TIME	NOT NULL, CHECK gio_dong > gio_mo
dang_hoat_dong	BOOLEAN	NOT NULL, default TRUE
tao_luc	TIMESTAMPTZ	NOT NULL

dang_hoat_dong = FALSE → chi nhánh tạm đóng, app khách báo "chi nhánh tạm nghỉ", không cho đặt đơn.

3.4.2 nguoi_dung — 3 vai trò

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
ho_ten	VARCHAR(100)	NOT NULL
sdt	VARCHAR(15)	NOT NULL, UNIQUE (tên đăng nhập)
email	VARCHAR(120)	UNIQUE, nullable
mat_khau_hash	VARCHAR(72)	NOT NULL — **bcrypt, không lưu mật khẩu thô**
vai_tro	VARCHAR(20)	NOT NULL, CHECK IN (CHU_CHUOI,QUAN_LY,KHACH)
chi_nhanh_id	BIGINT	FK → chi_nhanh, nullable
diem_tich_luy	INTEGER	NOT NULL default 0, CHECK ≥ 0
bi_khoa	BOOLEAN	NOT NULL default FALSE (khoá tài khoản)
tao_luc	TIMESTAMPTZ	NOT NULL

Ràng buộc chéo quan trọng:

- CHECK (vai_tro <> 'QUAN_LY' OR chi_nhanh_id IS NOT NULL) — quản lý ca **bắt buộc** gắn chi nhánh.
- CHECK (vai_tro NOT IN ('CHU_CHUOI','KHACH') OR chi_nhanh_id IS NULL) — chủ chuỗi và khách không gắn chi nhánh.

Đây là ràng buộc ở tầng CSDL, không chỉ ở code: dù backend có bug cũng không tạo được QUAN_LY "không thuộc chi nhánh nào".

3.4.3 nhom_mon — nhóm thực đơn

id PK · ten UNIQUE · thu_tu SMALLINT (thứ tự hiển thị) · dang_hien BOOLEAN (ẩn cả nhóm khỏi app khách).

3.4.4 mon — món gốc toàn chuỗi

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
nhom_mon_id	BIGINT	NOT NULL, FK → nhom_mon ON DELETE RESTRICT

Cột	Kiểu	Ràng buộc
ten	VARCHAR(120)	NOT NULL, UNIQUE trong nhóm
mo_ta	TEXT	
anh_url	TEXT	CHECK bắt đầu bằng http(s):// — **chỉ URL, không base64**
dang_ban	BOOLEAN	NOT NULL default TRUE (ngừng bán toàn chuỗi)

Giá **không** nằm ở đây. Món không có giá toàn chuỗi vì mỗi chi nhánh bán giá khác nhau.

3.4.5 mon_chi_nhanh — giá & hết hàng theo chi nhánh

Cột	Kiểu	Ràng buộc
mon_id	BIGINT	PK, FK → mon ON DELETE CASCADE
chi_nhanh_id	BIGINT	PK, FK → chi_nhanh ON DELETE CASCADE
gia_co_ban	BIGINT	NOT NULL, CHECK 0..10.000.000
het_hang	BOOLEAN	NOT NULL default FALSE

Khoá chính ghép (mon_id, chi_nhanh_id). **Không có dòng = chi nhánh không bán món đó** (tự động ẩn khỏi thực đơn). Đây là bảng quyết định giá thật của đơn hàng.

3.4.6 nhom_tuy_chon / tuy_chon

nhom_tuy_chon: id PK, ten UNIQUE (Size / Mức đường / Mức đá), bat_buoc BOOLEAN, chon_nhieu BOOLEAN, thu_tu.

- chon_nhieu = FALSE + bat_buoc = TRUE → khách **phải chọn đúng 1** (mỗi món chỉ 1 size). Server kiểm tra bằng bảng mon_tuy_chon.
- chon_nhieu = TRUE → cho chọn nhiều (ví dụ "Mức đá" nếu muốn cho phép).

tuy_chon: id PK · nhom_tuy_chon_id FK ON DELETE CASCADE · ten · phu_thu BIGINT CHECK 0..1.000.000 (size L +6.000đ, đường/đá = 0) · mac_dinh BOOLEAN · dang_dung BOOLEAN. UNIQUE (nhom_tuy_chon_id, ten).

3.4.7 mon_tuy_chon — món ↔ nhóm tuỳ chọn

PK (mon_id, nhom_tuy_chon_id), cả hai FK ON DELETE CASCADE. Ví dụ bánh mì không gán nhóm "Mức đá" → app khách không hiện lựa chọn đó.

3.4.8 topping / mon_topping

topping: id PK · ten UNIQUE · gia BIGINT CHECK 0..1.000.000 · dang_dung.

mon_topping: PK (mon_id, topping_id), FK CASCADE. Server chỉ chấp nhận topping có trong bảng này cho món tương ứng — chặn client gửi topping lạ.

3.4.9 ban — bàn & mã QR

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
chi_nhanh_id	BIGINT	NOT NULL, FK → chi_nhanh RESTRICT
ten_ban	VARCHAR(30)	NOT NULL, UNIQUE trong chi nhánh
ma_qr	VARCHAR(64)	NOT NULL, **UNIQUE**, default encode(gen_random_bytes(16),'hex')
dang_dung	BOOLEAN	NOT NULL default TRUE

ma_qr = 16 byte ngẫu nhiên do pgcrypto sinh → 32 ký tự hex, **không đoán được, không tuần tự**. Mất QR thì update ma_qr = default, QR cũ chết ngay.

3.4.10 ca_lam — mở ca / chốt ca

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
chi_nhanh_id	BIGINT	NOT NULL, FK RESTRICT
nguoi_dung_id	BIGINT	NOT NULL, FK RESTRICT
tien_dau_ca	BIGINT	NOT NULL default 0, CHECK ≥ 0
mo_luc	TIMESTAMPTZ	NOT NULL default now()
dong_luc	TIMESTAMPTZ	nullable, CHECK ≥ mo_luc
doanh_thu_ca	BIGINT	snapshot lúc chốt
tien_mat_ky_vong	BIGINT	snapshot lúc chốt
tien_mat_thuc_te	BIGINT	CHECK ≥ 0
tien_ck_thuc_te	BIGINT	CHECK ≥ 0
tien_chenh_lech	BIGINT	**cho phép âm** (thiếu tiền)
trang_thai	VARCHAR(10)	CHECK IN (DANG_MO,DA_DONG)
ghi_chu	TEXT	

Ràng buộc mạnh:

- CHECK (trang_thai = 'DANG_MO' OR dong_luc IS NOT NULL) — ca đã đóng phải có giờ đóng.
- UNIQUE INDEX ... (chi_nhanh_id) WHERE trang_thai = 'DANG_MO' — **mỗi chi nhánh chỉ 1 ca mở**, chặn ở tầng DB, không phụ thuộc code.
- UNIQUE INDEX ... (nguoi_dung_id) WHERE trang_thai = 'DANG_MO' — một nhân viên không mở 2 ca.

3.4.11 ma_giam_gia

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
ma	VARCHAR(20)	NOT NULL, UNIQUE
loai_giam	VARCHAR(10)	CHECK IN (PHAN_TRAM,SO_TIEN)
gia_tri	BIGINT	CHECK > 0; nếu PHAN_TRAM thì ≤ 100
don_toi_thieu	BIGINT	NOT NULL default 0, CHECK ≥ 0
giam_toi_da	BIGINT	nullable, chỉ dùng cho PHAN_TRAM
tu_luc / den_luc	TIMESTAMPTZ	NOT NULL, CHECK den_luc > tu_luc
so_luot_toi_da	INTEGER	NOT NULL, CHECK > 0
da_dung	INTEGER	NOT NULL default 0, CHECK 0 ≤ da_dung ≤ so_luot_toi_da
chi_nhanh_id	BIGINT	FK, **NULL = toàn chuỗi**
dang_bat	BOOLEAN	NOT NULL default TRUE

CHECK (da_dung <= so_luot_toi_da) là chốt chặn cuối: kể cả khi code sai, không thể vượt lượt.

3.4.12 don_hang

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
ma_don	VARCHAR(20)	NOT NULL UNIQUE, default HD + ngày + số thứ tự
ma_client	UUID	UNIQUE — chống tạo đơn trùng khi mất mạng
chi_nhanh_id	BIGINT	NOT NULL, FK RESTRICT
ban_id	BIGINT	NOT NULL, FK RESTRICT
khach_id	BIGINT	FK → nguoi_dung, **NULL = khách vãng lai**
ca_lam_id	BIGINT	FK → ca_lam (ca đang mở lúc tạo đơn)
ma_giam_gia_id	BIGINT	FK, nullable
trang_thai	VARCHAR(10)	NOT NULL default CHO_PHA, CHECK IN 4 trạng thái
tong_tien_hang	BIGINT	NOT NULL, CHECK ≥ 0
tien_giam	BIGINT	NOT NULL default 0, CHECK ≥ 0
thanh_tien	BIGINT	NOT NULL
phuong_thuc_tt	VARCHAR(15)	CHECK IN (TIEN_MAT,CHUYEN_KHOAN)
ly_do_huy	VARCHAR(255)	bắt buộc khi HUY
tao_luc / hoan_tat_luc / huy_luc	TIMESTAMPTZ	

Bất biến kế toán ở tầng DB:

```
CHECK (thanh_tien = tong_tien_hang - tien_giam AND thanh_tien >= 0)
CHECK (trang_thai <> 'HOAN_TAT' OR hoan_tat_luc IS NOT NULL)
CHECK (trang_thai <> 'HUY' OR (huy_luc IS NOT NULL AND ly_do_huy IS NOT NULL))
```

Không có cột giá nào do client gửi — backend tính rồi ghi.

3.4.13 chi_tiet_don — snapshot

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
don_hang_id	BIGINT	NOT NULL, FK ON DELETE CASCADE
mon_id	BIGINT	NOT NULL, FK RESTRICT
ten_mon	VARCHAR(120)	NOT NULL — **snapshot**
don_gia	BIGINT	NOT NULL — **snapshot giá gốc lúc đặt**
phu_thu	BIGINT	NOT NULL default 0 — tổng phụ thu **1 đơn vị**
so_luong	SMALLINT	CHECK 1..99
thanh_tien	BIGINT	CHECK = (don_gia + phu_thu) * so_luong

Vì có ten_mon + don_gia riêng, sửa giá menu hôm nay **không** làm đổi hoá đơn hôm qua. Vẫn giữ mon_id FK để thống kê top món.

3.4.14 chi_tiet_don_tuy_chon

Cột	Kiểu	Ràng buộc
id	BIGSERIAL	PK
chi_tiet_don_id	BIGINT	NOT NULL, FK ON DELETE CASCADE

Cột	Kiểu	Ràng buộc
tuy_chon_id	BIGINT	FK → tuy_chon ON DELETE SET NULL, nullable
topping_id	BIGINT	FK → topping ON DELETE SET NULL, nullable
ten_snapshot	VARCHAR(60)	NOT NULL
phu_thu_snapshot	BIGINT	NOT NULL default 0, CHECK ≥ 0

- CHECK ((tuy_chon_id IS NULL) <> (topping_id IS NULL)) — mỗi dòng thuộc **đúng một** loại (XOR). Một bảng phục vụ cả size/đường/đá và topping, tránh 2 bảng gần giống nhau.
- UNIQUE có điều kiện: mỗi dòng món chỉ chọn 1 lần mỗi tuỳ chọn / mỗi topping → chặn "2 size" ở tầng DB.
- ON DELETE SET NULL + snapshot: xoá topping khỏi menu vẫn giữ được tên/giá trong đơn cũ.

3.4.15 danh_gia

id PK · don_hang_id FK CASCADE · khach_id FK CASCADE · mon_id FK CASCADE · so_sao SMALLINT CHECK 1..5 · nhan_xet VARCHAR(500) · tao_luc.

UNIQUE (don_hang_id, mon_id) — một món trong một đơn chỉ đánh giá 1 lần.

3.4.16 lich_su_diem — sổ cái điểm

id PK · khach_id FK CASCADE · don_hang_id FK SET NULL · thay_doi INTEGER CHECK ≠ 0 · ly_do · tao_luc.

nguoi_dung.diem_tich_luy là số dư; bảng này là lịch sử để đối soát khi khách thắc mắc.

3.4.17 don_hang_log — vết trạng thái

id PK · don_hang_id FK CASCADE · tu_trang_thai VARCHAR(10) · den_trang_thai NOT NULL CHECK · nguoi_dung_id FK SET NULL · ghi_chu · tao_luc.

Dùng để tính thời gian pha trung bình và truy vết "ai huỷ đơn".

3.5. Index và lý do

Index	Phục vụ truy vấn
idx_don_hang_chi_nhanh_tt (chi_nhanh_id, trang_thai, tao_luc DESC)	App admin poll đơn CHO_PHA mỗi 3s — nóng nhất hệ thống
idx_don_hang_hoan_tat (chi_nhanh_id, hoan_tat_luc) WHERE trang_thai='HOAN_TAT'	Báo cáo doanh thu theo ngày/tuần
idx_don_hang_tao_luc	Lọc theo khoảng thời gian
idx_don_hang_khach	Lịch sử đơn của khách
idx_don_hang_ca_lam	Chốt ca: cộng tiền theo ca
idx_chi_tiet_don_mon	Top 10 món bán chạy
idx_chi_tiet_don_don_hang	Lấy chi tiết 1 đơn
ux_ca_lam_mo_theo_chi_nhanh (partial unique)	Chặn 2 ca mở cùng chi nhánh
idx_nguoi_dung_chi_nhanh / vai_tro	Danh sách nhân viên theo chi nhánh
idx_mon_chi_nhanh_chi_nhanh	Load thực đơn 1 chi nhánh
idx_danh_gia_mon	Điểm trung bình món
idx_lich_su_diem_khach	Lịch sử điểm

Không index cột trang_thai đơn lẻ vì độ chọn lọc thấp — luôn đi kèm chi_nhanh_id.

3.6. Nghiệp vụ dùng transaction

Một request POST /api/don-hang phải làm 4 việc "được ăn cả, ngã về không": tạo don_hang, tạo chi_tiet_don, tạo chi_tiet_don_tuy_chon, và trừ 1 lượt mã giảm giá. Thiếu transaction thì có thể ra đơn mà không trừ lượt (dùng mã vô hạn) hoặc trừ lượt mà không có đơn (mất lượt của khách).

```
// be/src/services/donHang.js
const client = await pool.connect();
try {
  await client.query('BEGIN');

  // (1) Khoá ca đang mở của chi nhánh.
  //     FOR UPDATE ngăn việc chốt ca chen giữa lúc đang tạo đơn.
  const { rows: [ca] } = await client.query(
    `SELECT id FROM ca_lam
      WHERE chi_nhanh_id = $1 AND trang_thai = 'DANG_MO'
      FOR UPDATE`,
    [chiNhanhId]
  );
  if (!ca) throw new AppError(409, 'CHUA_MO_CA');

  // (2) Đọc GIÁ THẬT từ DB. Bỏ qua hoàn toàn mọi field giá client gửi lên.
  const monIds = items.map(i => i.mon_id);
  const { rows: dsMon } = await client.query(
    `SELECT m.id, m.ten, m.dang_ban, mc.gia_co_ban, mc.het_hang
       FROM mon m
       JOIN mon_chi_nhanh mc
         ON mc.mon_id = m.id AND mc.chi_nhanh_id = $1
      WHERE m.id = ANY($2::bigint[])`,
    [chiNhanhId, monIds]
  );

  // (3) Kiểm tra + tính tiền (hàm thuần, không đụng DB)
  const { tongTienHang, dongMon } = tinhTien(dsMon, items);   // ném 400 nếu món hết hàng / không bán

  // (4) Tính giảm giá — cũng từ DB
  let tienGiam = 0;
  if (maGiamGiaId) {
    tienGiam = await tinhGiamGia(client, maGiamGiaId, tongTienHang);
  }

  // (5) Tạo đơn
  const { rows: [don] } = await client.query(
    `INSERT INTO don_hang
       (ma_client, chi_nhanh_id, ban_id, khach_id, ca_lam_id, ma_giam_gia_id,
        tong_tien_hang, tien_giam, thanh_tien, phuong_thuc_tt, ghi_chu)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11)
     RETURNING id, ma_don, thanh_tien`,
    [maClient, chiNhanhId, banId, khachId, ca.id, maGiamGiaId,
     tongTienHang, tienGiam, tongTienHang - tienGiam, phuongThucTt, ghiChu]
  );

  // (6) Chi tiết đơn + snapshot tuỳ chọn
  for (const d of dongMon) {
    const { rows: [ct] } = await client.query(
      `INSERT INTO chi_tiet_don
         (don_hang_id, mon_id, ten_mon, don_gia, phu_thu, so_luong, thanh_tien, ghi_chu)
       VALUES ($1,$2,$3,$4,$5,$6,$7,$8) RETURNING id`,
      [don.id, d.mon_id, d.ten_mon, d.don_gia, d.phu_thu, d.so_luong, d.thanh_tien, d.ghi_chu]
    );
    for (const tc of d.tuy_chon) {
      await client.query(
        `INSERT INTO chi_tiet_don_tuy_chon
           (chi_tiet_don_id, tuy_chon_id, topping_id, ten_snapshot, phu_thu_snapshot)
         VALUES ($1,$2,$3,$4,$5)`,
        [ct.id, tc.tuy_chon_id, tc.topping_id, tc.ten, tc.phu_thu]
      );
    }
  }

  // (7) Trừ lượt mã — UPDATE có điều kiện, an toàn khi 2 khách tranh lượt cuối
  if (maGiamGiaId) {
    const upd = await client.query(
      `UPDATE ma_giam_gia
         SET da_dung = da_dung + 1
       WHERE id = $1
         AND dang_bat
         AND now() BETWEEN tu_luc AND den_luc
         AND da_dung < so_luot_toi_da
       RETURNING id`,
      [maGiamGiaId]
    );
    if (upd.rowCount === 0) throw new AppError(400, 'MA_HET_LUOT');
  }
```

```
  await client.query('COMMIT');
  return don;
} catch (err) {
  await client.query('ROLLBACK');   // huỷ toàn bộ: không đơn, không trừ lượt
  throw err;
} finally {
  client.release();
}
```

Điểm cần nói khi bảo vệ:

- rowCount === 0 ở bước 7 xảy ra khi một request khác vừa dùng lượt cuối → ROLLBACK xoá luôn đơn vừa insert ở bước 5. Không có chuyện "đơn tạo rồi mà mã không trừ".
- FOR UPDATE ở bước 1 chặn race condition với "chốt ca": ca không thể bị đóng trong lúc đơn đang được ghi vào ca đó.
- Giá đọc lại từ DB **bên trong** transaction — kể cả khi admin đang sửa giá ở tab khác.

Transaction thứ hai (ngắn) — hoàn tất đơn + cộng điểm:

```
BEGIN;
UPDATE don_hang
   SET trang_thai = 'HOAN_TAT', hoan_tat_luc = now()
 WHERE id = $1 AND trang_thai = 'DANG_PHA'
RETURNING id, thanh_tien, khach_id;        -- 0 dòng => transition sai, ROLLBACK

-- chỉ chạy nếu đơn có khách
UPDATE nguoi_dung SET diem_tich_luy = diem_tich_luy + $diem WHERE id = $khach;
INSERT INTO lich_su_diem (khach_id, don_hang_id, thay_doi, ly_do)
VALUES ($khach, $1, $diem, 'Hoàn tất đơn ' || $ma_don);

INSERT INTO don_hang_log (don_hang_id, tu_trang_thai, den_trang_thai, nguoi_dung_id)
VALUES ($1, 'DANG_PHA', 'HOAN_TAT', $quanLyId);
COMMIT;
```

4. Bảng phân công (bản nháp)

Bản nháp nộp ở mốc tuần 9; cả nhóm họp chốt lại tỷ lệ đóng góp trước khi ký. Nguyên tắc: **không ai sửa file của người khác**; cần đổi hợp đồng API thì sửa docs/api-contract.md và báo cả nhóm.

4.1. Vai trò và tỷ lệ đóng góp dự kiến

Người	Vai trò	Phụ trách chính	%
P1	Backend core + CSDL	schema.sql, seed.sql, JWT + bcrypt, phân quyền middleware, **engine tính tiền**, tạo đơn (transaction), state machine đơn, bàn/QR, ca làm, đánh giá, điểm tích luỹ	24
P2	App Admin (Android)	Đăng nhập theo vai trò, quản lý thực đơn + bật/tắt hết hàng, danh sách đơn (polling 3 giây + âm báo), quản lý bàn & QR, mở–chốt ca	26
P3	App Client (Flutter)	Quét QR, thực đơn, tuỳ chọn món, giỏ hàng, mã giảm giá, đặt đơn, theo dõi trạng thái, đăng ký/đăng nhập, lịch sử + điểm, đánh giá	24
P4	Backend nghiệp vụ + màn hình báo cáo + QA/Deploy	API báo cáo / khuyến mãi / nhân viên và **các màn hình Admin tương ứng** (biểu đồ MPAndroidChart), deploy Render, Postman, test, báo cáo & video	26
Tổng			**100**

Ghi chú: P4 phụ trách cả các màn hình Admin cho báo cáo, khuyến mãi và nhân viên (code trong các Fragment riêng, không sửa file của P2) để đáp ứng yêu cầu app Admin có biểu đồ hoặc bảng báo cáo.

4.2. Phân công theo module

Module	Backend (API + SQL)	App Admin	App Client
Xác thực, phân quyền	P1	P2	P3
Thực đơn, giá theo chi nhánh, hết hàng	P1	P2	P3 (xem thực đơn)
Đặt đơn + tính tiền + áp mã giảm giá	P1	—	P3
Trạng thái đơn, theo dõi, âm báo đơn mới	P1	P2	P3
Bàn & mã QR	P1	P2	P3 (quét QR)
Ca làm (mở / chốt ca)	P1	P2	—
Báo cáo: doanh thu, top 10 món, giờ cao điểm	P4	P4	—
Quản lý mã giảm giá	P4	P4	P3 (nhập mã)
Quản lý nhân viên, số ca đã làm	P4	P4	—
Đánh giá món, điểm tích luỹ	P1	—	P3
Deploy Render, Postman, test, README, api.md	P4 (P1 hỗ trợ)	—	—
Báo cáo đồ án, video demo, slide	Mỗi người viết/quay phần của mình; P4 tổng hợp		

4.3. Lịch làm việc theo mốc của đề

Tuần	Mốc	P1	P2	P3	P4
8	Đăng ký đề	Tạo repo (3 thư mục backend/, admin-android/, client-flutter/), .gitignore có .env ngay commit đầu	Dựng project Android, Retrofit skeleton	Dựng project Flutter, màn mock	Danh sách endpoint, Postman rỗng
9	**Chốt phân tích** (tài liệu này)	Chạy schema.sql trên Supabase, /health	Thiết kế màn hình Admin	Thiết kế màn hình Client	Chốt docs/api-contract.md

Tuần	Mốc	P1	P2	P3	P4
10	Chuẩn bị mốc giữa kỳ	Auth, menu, engine tính tiền	Login thật + 2 màn hình	Menu + tuỳ chọn món (2 màn hình)	**Deploy Render sớm**, ≥ 3 API thật gọi được
11	**Giữa kỳ (1,0 điểm, không nộp bù)**	Hoàn thiện API đã có	Hoàn thiện 2 màn hình	Hoàn thiện 2 màn hình	Kiểm tra link API từ máy khác
12		Tạo đơn + state machine, ca làm	Quản lý thực đơn, danh sách đơn polling	QR → giỏ → đặt đơn	API báo cáo, khuyến mãi
13	**Bản chạy được**	Đánh giá, điểm, bàn/QR	Bàn/QR, mở–chốt ca, âm báo	Theo dõi đơn, đăng ký/đăng nhập, lịch sử, đánh giá	Màn Admin báo cáo/khuyến mãi/nhân viên, seed 60 đơn
14	Feature freeze	Fix bug, index, kiểm tra SQL	Fix bug, APK release	Fix bug, APK release, flutter analyze	E2E test, Postman, báo cáo + video
15	**Bảo vệ**	Dự phòng	Dự phòng	Dự phòng	Bật sẵn dịch vụ, nộp đủ mục D.1

4.4. Quy ước làm việc nhóm

- Mỗi thành viên dùng **tài khoản GitHub riêng**, cấu hình user.email đúng; tối thiểu **15 commit có ý nghĩa** rải đều 8 tuần. Thông điệp commit mô tả đúng việc đã làm (ví dụ “them API dat don co kiem tra mon het hang”), không dùng “update”, “fix”, “abc”.
- Nhánh git feat/<ten>-<viec>, PR nhỏ; main chỉ merge qua PR, P4 review.
- .env nằm trong .gitignore từ commit đầu tiên; không commit khoá service_role của Supabase.
- Backend kết nối Supabase bằng **Connection pooler, Session mode, cổng 5432** (không dùng Direct connection vì Render chỉ có IPv4); pg.Pool đặt max: 5, ssl: { rejectUnauthorized: false }.
- Render gói Free ngủ sau 15 phút: gọi thử /health 2–3 phút trước khi demo; vào Supabase khôi phục dự án nếu bị tạm dừng sau 7 ngày không hoạt động.
- Tên app, tên gói và TAG log chứa mã nhóm <MA_NHOM>.
- Dùng AI để sinh mã là được phép, nhưng mỗi người phải giải thích được từng dòng mình commit khi bảo vệ.

4.5. Xác nhận của nhóm

Họ tên	MSSV	Vai trò	% đóng góp	Chữ ký
		P1 – Backend core + CSDL	24	
		P2 – App Admin	26	
		P3 – App Client	24	
		P4 – Backend nghiệp vụ + báo cáo + QA/Deploy	26	